

```

distance1 = howfar(genlines(cities, itinerary))
distance2 = howfar(genlines(cities, itinerary2))

itinerarytoreturn = itinerary.copy()

randomdraw = np.random.rand()
temperature = 1/((time/1000) + 1)

if((distance2 > distance1 and (randomdraw) < (temperature)) or (distance1 >
distance2)):
    itinerarytoreturn=itinerary2.copy()

return(itinerarytoreturn.copy())

```

***Listing 6-4:** An updated version of our `perturb()` function that takes into account the temperature and a random draw*

Just by adding those two short lines, a new argument, and a new `if` condition (all shown in bold in [Listing 6-4](#)), we already have a very simple simulated annealing function. We also changed the temperature function a little; because we'll be calling this function with very high `time` values, we use `time/1000` instead of `time` as part of the denominator argument in our temperature function. We can compare the performance of simulated annealing with the `perturb` search algorithm and the nearest neighbor algorithm as follows:

```

itinerary =
[0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,
\30,31,32,33,34,35,36,37,38,39]
np.random.seed(random_seed)

itinerary_sa = itinerary.copy()
for n in range(0, len(itinerary) * 50000):
    itinerary_sa = perturb_sa1(cities, itinerary_sa, n)

print(howfar(genlines(cities, itinerary))) #random itinerary
print(howfar(genlines(cities, itinerary_ps))) #perturb search
print(howfar(genlines(cities, itinerary_sa))) #simulated annealing
print(howfar(genlines(cities, donn(cities, N)))) #nearest neighbor

```
